

Node.js + React — Priority 20 Flashcards

Cover the answer structure, recall it out loud, then check yourself against the card.

Master trick: most of these follow M.I.F.V. — **Measure**/confirm → **Investigate** the specific cause → **Fix** → **Validate** under real load. Learn that pattern once and Cards 2, 8, 13, 14, 19 become near-automatic.

Card 1

Laerdal — Explain a system you built, a production issue, and how you'd scale it.

Memory trick: Framework: Architecture → Issue → Investigation → Fix → Bottleneck-first scaling → Validate

Answer structure: Architecture + traffic → real issue + investigation → fix → identify bottleneck (CPU/mem/event-loop/DB/pool) → scale that specific thing → validate with load test + p95/p99.

Card 2

Laerdal — How do you detect a memory leak in Node.js?

Memory trick: M.I.F.V. + heap snapshot diff

Answer structure: Confirm real leak (RSS/heapUsed after GC) → 2 heap snapshots, diff retained objects → check caches/closures/listeners/timers → fix → re-verify under load.

Card 3

Laerdal — How would you increase API performance?

Memory trick: Measure first: p50/p95/p99 + tracing to split Node/DB/cache/downstream

Answer structure: Measure p95/p99 + tracing to split time across Node/DB/cache/downstream → fix the ACTUAL bottleneck (query, sync code, or downstream timeout) → validate with load test.

Card 4

Laerdal — How would you handle a bulk response?

Memory trick: Question the need first, then pagination/streaming, then virtualization on frontend

Answer structure: Ask if full dataset is needed → pagination/filtering/streaming → trim fields + compress/cache → frontend virtualization instead of rendering everything.

Card 5

Laerdal — What happens if `process.nextTick()` is used heavily?

Memory trick: Starves the event loop — timers/I-O/requests get delayed

Answer structure: Runs before event loop's next phase → heavy/recursive use starves the loop → timers, I/O, requests get delayed → move CPU-heavy work to Worker Threads instead.

Card 6

Laerdal — How do you handle graceful shutdown?

Memory trick: SIGTERM → stop new traffic → finish in-flight → close connections → exit

Answer structure: On SIGTERM: stop new traffic → let in-flight requests finish (with timeout) → stop new jobs → close DB/external connections → exit; readiness checks flip first in k8s.

Card 7

Laerdal — File-processing feature: design for performance and scale?

Memory trick: Stream (don't load full file) → Worker Threads for CPU work → queue → monitor

Answer structure: Stream + backpressure (no full file in memory) → CPU-heavy work off to Worker Threads/background worker → fronted by a queue → monitor queue depth/failures → scale workers independently.

Card 8

Noccarc — Memory leak / heap out-of-memory — how do you handle it?

Memory trick: Same M.I.F.V. + streams/backpressure for large payloads

Answer structure: Confirm genuine leak vs workload rise → snapshot compare → check caches/closures/listeners/timers → streams+backpressure if large files → fix root cause, not just heap-size bump.

Card 9

Noccarc — How would you optimize large-list rendering in React?

Memory trick: Profile first (DevTools) → virtualization → stable keys/prop refs → memo only if profiling proves it

Answer structure: Profile with React DevTools → virtualization/pagination for thousands of rows → check stable keys + prop reference changes → React.memo/useMemo only after profiling proves benefit → server-side pagination for very large data.

Card 10

Neuro Spark Works — Optimize a MERN app for performance?

Memory trick: One end-to-end problem: Mongo indexes + Node profiling + React Profiler + measure before/after

Answer structure: MongoDB: slow queries/indexes/unnecessary fields. Node: profile endpoints, sync ops, pooling. React: Profiler, virtualization. Plus payload/caching/compression. Measure before AND after.

Card 11

G2i — What could cause React performance bottlenecks?

Memory trick: 4 buckets: Rendering, JS execution, Network, DOM size

Answer structure: Split into rendering (Profiler), JS execution, network (payload/API), DOM size → check cascading re-renders and fresh prop references → fix actual bottleneck, don't reflexively memoize.

Card 12

Public report — Investigate a sudden spike in API latency?

Memory trick: When/scope → compare metrics → trace across layers → check deploys → rollback if severe → verify + alert

Answer structure: When did it start, all endpoints or one? → compare p50/p95/p99 vs volume/CPU/error → trace across Node/DB/Redis/downstream → check recent deploys → rollback if severe → verify + add alerting.

Card 13

Public report — How do you safely deploy without downtime?

Memory trick: Rolling/blue-green/canary + graceful shutdown + readiness checks

Answer structure: Rolling/blue-green/canary depending on infra → old instance stops new traffic, drains via graceful shutdown → readiness/liveness checks → canary for risky changes → rollback fast if metrics degrade.

Card 14

Coditas — App slow after a deployment — how do you debug?

Memory trick: Compare to last healthy version → what changed → trace → rollback if severe → validate

Answer structure: Compare new vs last healthy (error rate, p95/p99, CPU, memory, event-loop) → identify what actually changed → trace to pinpoint → rollback/flag-off if severe → validate with prod-like load test.

Card 15

Coditas — How do you identify blocking operations in Node.js?

Memory trick: Sync FS/JSON/crypto/CPU loops → correlate CPU with event-loop lag → profile → move to Worker Threads

Answer structure: Look for sync filesystem calls, large JSON parse/stringify, CPU loops, sync crypto → correlate CPU usage with event-loop lag → profile to find exact function → move to Worker Threads/separate process.

Card 16

Pearson — Redis eviction policies — which and why?

Memory trick: LRU = recency, LFU = frequency, plus expiry-only policies + TTLs always

Answer structure: LRU (recency-based) vs LFU (frequency-based) vs expiry-restricted policies → choose LFU for hot frequently-hit data, LRU for recency-driven access → always set sensible TTLs too.

Card 17

Pearson — How can Redis be used as a Pub/Sub manager?

Memory trick: Pub/Sub = real-time but NOT durable → Streams/Kafka for durability

Answer structure: Publishers → channels → subscribers, good for real-time notifications, but NOT durable — disconnected subscribers miss messages; use Redis Streams or Kafka when durability/replay is needed.

Card 18

Pearson — Worker Thread under high CPU load — how to terminate?

Memory trick: Check stuck vs expected work → Worker API terminate() → bound the pool with timeouts/limits

Answer structure: Check if it's expected heavy work or genuinely stuck (monitor execution time) → use Worker API's terminate() → bound CPU jobs with timeouts, cancellation, queue limits, bounded pool size.

Card 19

Cognizant — How do you identify a failure/performance issue in production?

Memory trick: Observability first → blast radius → correlate with deploys/config/traffic → mitigate → root cause → validate

Answer structure: Start with logs/metrics/traces → determine blast radius across rate/error/latency/CPU/memory/DB → correlate with deploys/config/traffic → mitigate customer impact first → then root cause → validate fix.

Card 20

Klarna — Find the memory leak in this code (live-code style).

Memory trick: Look for things that outlive their scope: listeners, intervals, subscriptions, unclosed closures

Answer structure: Look for resources outliving their component/operation — listeners, intervals, subscriptions, async work not cleaned up → check useEffect cleanup → reproduce via mount/unmount cycles → confirm with memory tools.
